

Understanding the Flower Season Data Transformation Logic in Python

Big Picture

Your original dataset contains one row per flower, such as: rose | all season tulip | march But charts need one row per month. So the code transforms one flower row into multiple monthly rows.

Step 1: Create Empty Basket

Code: `rows = []`

This creates an empty list called rows. Think of it like an empty basket where transformed rows will be stored.

Step 2: Loop Through Dataset

Code: `for _, row in df.iterrows():`

`iterrows()` means: go through the dataframe row by row. The variable row represents ONE flower row at a time.

Example: First loop -> rose row Second loop -> tulip row

What Does ' _ ' Mean?

`iterrows()` returns two things: (index, row) Example: (0, rose row) (1, tulip row) We do not need the index number, so ' _ ' means "ignore this value".

Step 3: Get Season

Code: `season = str(row["blooming_season"]).lower()`

This gets the blooming season from the current flower row and converts it into lowercase text. Example: "March" becomes "march".

Step 4: Check If All Season

Code: `if season == "all season":`

This asks: "Is this flower available all year?" If yes, the code creates rows for January through December.

Step 5: Create 12 Months

Code: `for m in range(1, 13):`

This loop creates numbers 1 through 12 representing months. Example: 1 = January 2 = February ... 12 = December

Step 6: Copy Row

Code: `new_row = row.copy()`

This creates a duplicate of the flower row so it can be safely modified without changing the original row.

Step 7: Create Date

Code: `new_row["date"] = f"2026-{str(m).zfill(2)}-01"`

`zfill(2)` makes the month two digits. Example: 1 becomes 01 Final result: 2026-01-01

Step 8: Save New Row

Code: `rows.append(new_row)`

This adds the transformed row into the rows basket.

Else Block: Single Month Flowers

If the flower is NOT all season, the code enters the else block. Example: tulip | march

Step 9: Unknown Month Number

Code: `month_num = None`

None means: "We do not know the month number yet."

Step 10: Search Month Map

Code: `for key in month_map:`

`month_map` is a dictionary like: jan -> 01 feb -> 02 mar -> 03 The loop checks month names one by one.

Step 11: Match Month Text

Code: `if key in season:`

Example: "mar" in "march" -> True This helps Python identify which month is being referenced.

Step 12: Translate Month

Code: `month_num = month_map[key]`

This translates: mar -> 03 using the dictionary.

Step 13: Break

Code: break

Once the month is found, stop looping because there is no need to continue searching.

Step 14-16: Create Final Row

The code copies the row again, creates the date, and appends it into the final transformed dataset. Example: tulip | march becomes: tulip | 2026-03-01

Final Result

The transformed dataset becomes timeline-ready for visualizations and charts. Example output: rose | 2026-01-01
rose | 2026-02-01 ... tulip | 2026-03-01